LightSense 频闪测量仪 · 初学者知识学习与项目搭建培训教程

> **课程定位**：面向嵌入式初学者，以 LightSense 项目（ESP32-S3 光频闪测量仪）为载体，系统学习 信号采集、数字信号处理、嵌入式外设驱动、Web 交互与数据持久化，并亲手完成整个项目的搭建与联调。
>
> **课程规模**：10 课时 × 30 分钟（知识讲解约 15 分钟 + 动手实操约 12 分钟 + 小结思考约 3 分钟）
>
> **先修要求**：基础 C/C++ 语法；无需单片机与电路经验
>
> **结业标准**：独立完成硬件接线、固件编译烧录、全功能验证（频闪测量 / LED 光源 / OLED 按键 / 网页仪表 / 教室测量导出）

课程总览

课时	主题	核心知识	对应代码
1	项目概览与开发环境搭建	频闪的危害与标准、PlatformIO 工具链	仓库浏览
2	ESP32-S3 硬件平台与接线	GPIO/ADC/PWM/I2C 外设概念、引脚规划	`src/config.h`
3	工程结构与首次编译烧录	PlatformIO 配置、分区表、串口监视	`platformio.ini`
4	信号采集：ADC + DMA 连续采样	采样定理、DMA 块传输、缓冲余量设计	`adc_dma.*`
5	测量核心：FFT 与 Pst^LM / SVM	频谱分析、窗函数、标准指标计算	`flicker.*`
6	LED 标准闪烁源与 PWM	LEDC 外设、模式表设计、闭环自测	`led_pwm.*`
7	OLED 显示与按键交互	U8g2、软件/硬件 I2C 取舍、消抖与状态机	`oled.*` `keypad.*`
8	无线网页仪表	WiFi AP、异步服务器、WebSocket 推送	`webdash.*`
9	教室测量与数据管理	多灯具测量流程、NVS 持久化、CSV 导出	`classroom.*`
10	调试方法论与项目拓展	真实案例复盘、探针调试法、改动检查单	全项目

第 1 课 项目概览与开发环境搭建（30 min）

时间分配

环境认知 5 min → 频闪知识 10 min → 安装实操 12 min → 小结 3 min

学习目标

- 说清楚"光生物频闪"是什么、为什么要测量

- 理解项目的功能全貌与系统框图
- 完成 VS Code + PlatformIO 开发环境安装

知识讲解

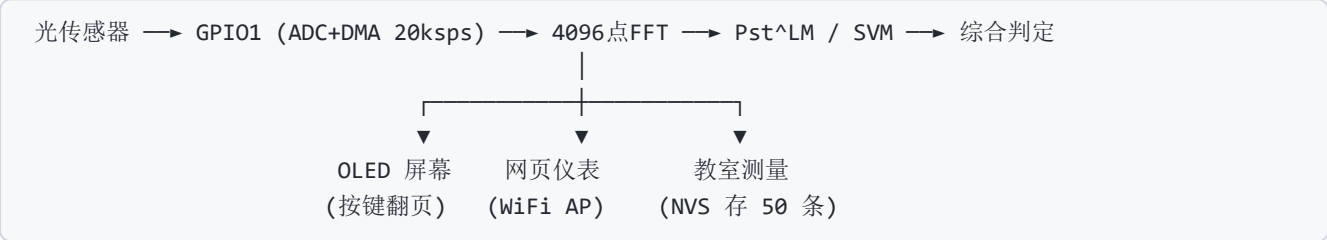
1. 什么是频闪?

光源的光通量随时间波动（闪烁）的现象。由市电整流、PWM 调光等产生。低频频闪可引起 视觉疲劳、头痛，甚至工作安全事故；高频波动虽不可见，也可能有生物影响。

2. 两大衡量指标（本项目完整实现）

- **Pst^LM（短期闪烁指示值）**：IEC 61000-4-15 定义闪烁计模型，IEC TR 61547-1 扩展到照明 产品。覆盖 **0~80 Hz**，合格线 Pst^LM ≤ 1。
- **SVM（频闪可见度度量）**：IEC TR 63158 定义，覆盖 **80~2000 Hz**。 SVM = (Σ (C_m / T(f_m))^3.7)^(1/3.7)，SVM = 1 表示 50% 人群刚好可见； 欧盟 (EU) 2019/2020 自 2024 年 9 月起要求 SVM ≤ 0.4。

3. 系统框图（先建立全貌，后续每课展开一块）



4. 开发工具链

- **VS Code**：编辑器
- **PlatformIO**：嵌入式开发插件，管理"平台（芯片框架）+ 库依赖 + 编译烧录"
- 本项目固定 `espressif32@6.10.0`（Arduino-ESP32 2.0.17）——**不要随意升级平台**， 原因见第 7 课案例

实操环节

1. 安装 VS Code 与 PlatformIO 插件（扩展市场搜索 "PlatformIO IDE"）
2. 克隆/拷贝 LightSense 仓库到本地，用 VS Code 打开
3. **重要（OneDrive 用户）**：若工程放在 OneDrive 同步目录，确认 `platformio.ini` 的 `[platformio]` 段已把 `build_dir` / `libdeps_dir` 重定向到非同步目录 （OneDrive 文件锁会让构建产物随机损坏，报"undefined reference"八成是它）
4. 浏览仓库： `src/` 下 9 组模块文件 + `readme.md`

检查点

- [ ] PlatformIO 插件安装完成，侧边栏出现蚂蚁图标
- [ ] 能说出 Pst^LM 与 SVM 各覆盖的频段与合格线

思考题

1. 为什么一个指标不够，要 Pst^LM 和 SVM 两个指标互补？
2. 你的工作目录在网盘同步范围内吗？构建目录为什么必须排除？

第 2 课 ESP32-S3 硬件平台与接线（30 min）

时间分配

芯片认知 8 min → 引脚规划 7 min → 接线实操 12 min → 小结 3 min

学习目标

- 了解 ESP32-S3 的核心外设资源
- 掌握"选引脚要避开什么"的工程思维
- 完成全部硬件接线

知识讲解

1. 核心外设速览

外设	本项目用途	关键参数
ADC1 + DMA	光信号连续采集	12 bit，20 ksps 连续模式（仅 ADC1 支持 DMA）
LEDC (PWM)	LED 闪烁源	10 bit 分辨率，硬件保证时序
I2C	OLED 显示	本项目用 GPIO 位拍"软件 I2C"（第 7 课讲为什么）
GPIO 内部上拉	五键薄膜按键	输入模式 + 上拉，按下接地为低
NVS (Flash)	教室记录掉电保存	键值存储，50 条 × 74 B
WiFi	网页仪表	AP 热点模式，SSID `LightSense`

2. 引脚分配总表（最新版）

引脚	用途	说明
GPIO1 (ADC1_CH0)	光信号输入	12 dB 衰减，满量程约 3.1 V
GPIO17 (D7)	LED 闪烁源输出	经 220~330 Ω 限流电阻接 LED
GPIO21 (D8)	按键公共端	**固件常置低电平作"虚拟 GND"**（`KEY_GND_PIN`）
GPIO4 / GPIO5	OLED SDA / SCL	板载丝印 SDA/SCL，1.54" SSD1309
GPIO38~42	五键薄膜按键	上拉输入；S3 的 JTAG 默认走 USB 19/20，占用 38~42 无冲突

3. 选引脚的工程思维（避坑清单）

- **Strapping 引脚**（0/3/45/46 等）：上电瞬间电平决定启动模式，尽量不接外设
- **USB 引脚**（19/20）：USB 通信占用
- **Flash/PSRAM 引脚**（26~32 等）：存储总线
- JTAG 复用引脚（39~42）：默认 JTAG 走 USB，可安全用作按键
- 输出引脚：串联限流电阻，勿超过单脚 ~40 mA 的绝对极限

实操环节（对照引脚表接线）

1. 光传感器（光电二极管/光敏电阻分压）→ GPIO1，注意满量程约 3.1 V
2. LED：GPIO17 (D7) → 220~330 Ω → LED → GND
3. 薄膜按键模块：5 根信号线 → GPIO38/39/40/41/42，**公共端 → GPIO21 (D8)**（不是物理 GND！）
4. OLED：VCC→3.3 V，SDA→GPIO4，SCL→GPIO5，GND→GND
5. 万用表复查电源短路后再上电

检查点

- [ ] 能说出为什么按键公共端接 GPIO21 而不直接接 GND（接线便捷，固件拉低等效）
- [ ] 至少列出三类"不能随便用"的引脚

思考题

1. 为什么 ADC 连续采样只能用 ADC1？（提示：ADC2 与 WiFi 共存限制）
2. 若把 LED 直接接 GPIO 而不加限流电阻，可能发生什么？

第 3 课 工程结构与首次编译烧录（30 min）

时间分配

工程文件解读 10 min → 编译烧录 15 min → 小结 5 min

学习目标

- 读懂 `platformio.ini` 每一行的作用
- 理解三层软件架构与模块划分
- 完成首次编译、烧录与串口监视

知识讲解

1. `platformio.ini` 逐段解读

```
[env:wifiduino32s3]
platform = espressif32@6.10.0     ; 固定官方平台 (arduino-esp32 2.0.17), 代码基于 2.x API
board = wifiduino32s3             ; 板型 (Blinker WiFiduino32S3, 8MB Flash)
framework = arduino               ; Arduino 框架
board_build.partitions = default_8MB.csv   ; 8MB 分区表: app0 扩到 3.375MB
                                  ; (U8g2 全量中文字库曾把默认 1.25MB app 撑爆到 102.8%)
lib_deps =
    kosme/arduinoFFT              ; FFT 计算
    me-no-dev/ESPAsyncWebServer   ; 异步 Web 服务器
    me-no-dev/AsyncTCP
    olikraus/U8g2                 ; OLED 图形库
```

2. 三层软件架构（可移植性设计）

```
┌──────────── 编排层 ────────────┐
│ main.cpp: 初始化 + 主循环调度 │
└────────────────────────────────┘

┌ 测量层 ┐  ┌ 交互层 ──────────┐
│flicker │  │ webdash (网页)   │
│classroom│ │ oled + keypad    │
└────────┘  └──────────────────┘

┌──────────── 驱动层 ────────────┐
│ adc_dma │ led_pwm │ (I2C由U8g2管) │
└────────────────────────────────┘
```

主循环每圈：按键事件 → 取一块 ADC 数据 → `flickerProcessBlock()` → 满窗口 （5 块 ≈1.024 s）时 `flickerReport()` → 教室/OLED/网页各自消费快照。

3. 模块 API 速查（后面每课用到）

模块	关键函数
adc_dma	`adcDmaBegin()` `adcDmaReadBlock(float*)` `adcDmaTakeOverflow()`
flicker	`flickerProcessBlock()` `flickerReport()` `flickerVerdict(pst,svm,vavg)`
led_pwm	`ledPwmBegin()` `ledPwmSetMode(idx)` `ledPwmGetMode()`
oled/keypad	`oledBegin()` `oledShow(snap)` `oledHandleKey(ev)` `keypadBegin()` `keypadGetEvent()`
webdash	`webdashBegin()` `webdashBroadcast(snap)`
classroom	`classroomBegin()` `classroomStart(label,windows)` `classroomOnWindow(snap)` 等

实操环节

1. 命令行编译：`python -m platformio run -e wifiduino32s3` （首次会自动下载平台与库，耐心等待）
2. 烧录：`python -m platformio run -t upload --upload-port COM8` （COM 号在设备管理器查）

3. 串口监视：`python -m platformio device monitor`（115200 bps；打开串口会复位板子）
4. 手机/电脑连接 WiFi `LightSense`（密码 `lightsense`），浏览器访问 `http://192.168.4.1`

检查点

- [ ] 编译输出 RAM / Flash 占用率（正常约 45% / 40%）
- [ ] 网页能打开并看到实时数据

思考题

1. 为什么 Flash 占用率超 100% 时要换分区表而不是删功能?
2. `lib_deps` 里每个库分别解决什么问题?

第 4 课 信号采集：ADC + DMA 连续采样（30 min）

时间分配

采样理论 10 min → 代码走读 8 min → 实操观察 9 min → 小结 3 min

学习目标

- 掌握采样率/分辨率/带宽的关系
- 理解 DMA 块传输与"缓冲余量"设计
- 学会从串口 DBG 输出判断采样健康度

知识讲解

1. 本项目的采样参数（`config.h`）

参数	值	推导
采样率	20 ksps	分析带宽 = 采样率/2 = 10 kHz（覆盖 5 kHz SVM 上限有余量）
FFT 点数	4096	频率分辨率 = 20000/4096 ≈ **4.883 Hz**
一块时长	≈204.8 ms	4096 / 20000
报告窗口	5 块 ≈1.024 s	Pst 以 1 min 为标称周期，这里做 1 s 快速档

2. 为什么要 DMA?

20 ksps 若靠 CPU 逐点中断搬运，中断频率 20 kHz，主循环会被打断到无法运行 WiFi/OLED。 DMA 让 ADC 外设把样本自动写入内存块，CPU 只在整块就绪后取走——一次搬 4096 点。

3. 缓冲余量：一个真实的翻车案例

驱动内部缓冲原来是 2 块（≈410 ms 余量）。第 7 课会讲到换成软件 I2C 后每帧 OLED 绘制 耗时涨到 100~200 ms，主循环变慢，缓冲被写满后 IDF 4.4 的 ADC 驱动返回 `ESP_ERR_INVALID_STATE` **并就此停摆**——表现就是网页和频闪仪数据永久冻结。 修复 = 缓冲扩到 4 块（≈820 ms 余量）+ 检测到该错误码时 `adc_digi_stop()/adc_digi_start()` 自愈重启。

设计原则：外设驱动要有"比你最坏情况还慢的主循环"更长的缓冲余量，并具备错误自愈。

代码走读（`adc_dma.cpp`）

- `adcDmaBegin()`：配置 12 dB 衰减、20 ksps 连续 DMA，`max_store_buf_size` = 4 块
- `adcDmaReadBlock(float *out)`：取 1 块 → 转 float；遇到 `ESP_ERR_INVALID_STATE` → 计溢出计数 + stop/start 重启，本块返回 false（丢弃，等下块）
- `adcDmaTakeOverflow()`：供报告层取出溢出计数（每 2 s 打一行 `DBG ovr=`）

实操环节

1. 打开串口监视，找 `DBG ovr=` 行
2. 用手机闪光灯/台灯对传感器快速晃动制造负载，观察 ovr 是否偶有增长 （偶发属正常——丢 1 块不影响指标；持续增长才是问题）

检查点

- [ ] 能算出：采样率 20 ksps、4096 点时频率分辨率是多少
- [ ] 理解"块 (block)"与"报告窗口 (window)"两级结构

思考题

1. 如果想分析到 10 kHz 的频闪分量，采样率和 FFT 点数要怎么改?
2. 为什么丢一块数据对 Pst/SVM 结果几乎没有影响，而连续停摆却是灾难?

第 5 课 测量核心：FFT 与 Pst^LM / SVM（30 min）

时间分配

FFT 与窗函数 10 min → 指标计算 8 min → 代码走读与实操 9 min → 小结 3 min

学习目标

- 理解 FFT 频谱、Hann 窗的作用与"主瓣簇恢复"
- 掌握 Pst^LM / SVM 的计算思想与本项目实现
- 会解读综合判定结果

知识讲解

1. 时域 → 频域

波形是光强随时间的曲线；FFT 把它分解成各频率分量。频闪指标都定义在频域上： 波动深度、各频率的归一化幅度 C_m。

2. Hann 窗与"主瓣簇"（本项目踩过的最深的技术坑）

截取有限波形等效于乘矩形窗，频谱会泄漏。加 Hann 窗可抑制泄漏，但真实频率若不在 bin 中心，能量会摊到相邻 2~3 个 bin。 **本项目处理法**：对每个局部极大值取相邻 3 bin 幅度之和 ÷ 2 恢复真实幅度，再对 3 点做抛物线插值求真实频率——保证每个分量只被计一次、加权一次。 （教训：最初把同一簇重复计入平方和，指标虚高。）

3. Pst^LM（0~80 Hz，bin 1~16）

按 IEC 61000-4-15 闪烁计的频率加权曲线，对 0.5~25 Hz 最敏感；对每个分量 归一化幅度 C_m 加权后代入标准模型，Pst^LM ≤ 1 合格。

4. SVM（80~2000 Hz，bin 17~409）

- 可见度阈值 $T(f) = 2.865\times10^{-5}\cdot f^{1.543} + 0.225$（幂律拟合）
- SVM = (Σ (C_m/T(f_m))^3.7)^(1/3.7) —— 注意是 **3.7 次方 Minkowski 范数**， 不是平方和；敏感分量对结果贡献极大
- 锚点自检：100 Hz / 100% 调制的方波 SVM ≈ 4.9（实现后复算吻合）

5. 综合判定（v1.1 规则）

```
窗口均值 vavg < 400        → FV_DARK  (6) 光线过暗 (DC 基准太小, 噪声虚高)
窗口均值 vavg > 3000       → FV_BRIGHT(7) 光线太亮 (接近满量程, 可能削顶)
否则 Pst^LM ≤ 1 且 SVM ≤ 0.4 → PASS (0);  超哪个报哪个 (2/3/5)
```

先做光照前置否决再算指标——超出量程时指标本身无效，直接提示更准确。

代码走读（`flicker.cpp`）

- `flickerProcessBlock()`：累积 5 块样本 + 频谱幅度谱（specAccum）
- `flickerReport()`：加窗 → FFT → 主瓣簇提取 → 计算 Pst/SVM → 打印/广播快照
- `flickerVerdict()`：上面的判定逻辑，返回 `FV_*` 码

实操环节

1. LED 控制页切到 50 Hz，把 LED 对准传感器 → 串口应显示主频 ≈50 Hz
2. 改 100 Hz 再测；对比波动深度与 SVM 读数
3. 挡住光线（变暗）→ 观察判定变成"光线过暗"

检查点

- [ ] 能解释为什么不能直接对 FFT 结果逐 bin 求和（泄漏 + 重复计入）
- [ ] 知道 SVM 的 3.7 次方意味着"最差分量主导"

思考题

1. 为什么判定用窗口均值 vavg 而不是峰值 vmax?
2. 50% 调制的方波在 100 Hz 时，频谱上会出现哪些分量?

第 6 课 LED 标准闪烁源与 PWM（30 min）

时间分配

LEDC 原理 8 min → 模式设计 7 min → 闭环自测实操 12 min → 小结 3 min

学习目标

- 理解 LEDC 硬件 PWM 的频率/分辨率/占空比
- 学会用"标准闪烁源"反向验证测量系统

知识讲解

1. LEDC 硬件 PWM

ESP32-S3 的 LEDC 外设按时钟分频自动产生精确方波，**不占 CPU**——这正是 "标准闪烁源"需要的：频率准确性由硬件保证，不受 WiFi/绘图干扰。 本项目（Arduino-ESP32 2.x 通道式 API）：`ledcSetup(ch, freq, bits)` → `ledcAttachPin(pin, ch)` → `ledcWrite(ch, duty)`。 10 bit 分辨率 = 1024 级占空比。

2. 七种模式与模式表（`led_pwm.cpp`）

idx	模式	输出
0	常灭	0%
1	常亮	100%
2~6	50/100/500/1000/5000 Hz	50% 占空比方波

设计要点：闪烁模式用 LEDC；常灭/常亮则 `ledcDetachPin` 后 `digitalWrite` 直出 （0%/100% 用 PWM 反而绕弯）。模式表是"数据驱动"的典型写法——加一种模式 只改表，不改逻辑。

3. 测量与激励互不干扰

LED 输出与 ADC 采集彼此独立 → 可以把 LED 对准自家传感器，形成**闭环自测**： 已知激励（如 50 Hz/50% 方波），验证测得的主频/波动深度是否吻合。

实操环节（闭环自测）

1. 网页 LED 控制页切 50 Hz → 频闪仪页读主频、波动深度（应 ≈50 Hz / ≈100%）
2. 依次切 100 Hz / 1 kHz / 5 kHz，记录各档 Pst 与 SVM 变化
3. （思考观察）5 kHz 已超出 SVM 频段上限 → SVM 应接近 0；Pst 同理

检查点

- [ ] 能解释常亮为什么不用 LEDC 100% 占空比直出
- [ ] 完成一轮 5 档频率的闭环验证并记录

思考题

1. 50% 占空比的方波，其波动深度是多少？如果占空比改 10% 呢?
2. 想新增"呼吸灯"模式，模式表和切换逻辑要怎么扩展?

第 7 课 OLED 显示与按键交互（30 min）

时间分配

U8g2 与页面设计 8 min → SW/HW I2C 案例分析 7 min → 按键与实操 12 min → 小结 3 min

学习目标

- 掌握 U8g2 全缓冲绘图与页面状态机
- **通过真实案例理解软件 I2C 与硬件 I2C 的取舍**
- 掌握按键消抖与"单任务绘制"并发设计

知识讲解

1. U8g2 与三页状态机（`oled.cpp`）

- U8g2 全缓冲（F）模式：先画到内存帧，`sendBuffer()` 一次性推屏
- 中文字库 wqy12/wqy16（GB2312 全量，所以第 3 课要 8MB 分区表）
- 三页 = 与网页一致的 `s_page` 状态机：0 频闪仪 / 1 LED 控制 / 2 教室测量

2. 案例分析：为什么最终选"软件模拟 I2C"

现象：首次上电，固件卡死在 OLED 初始化，串口无后续输出。 **定位**：分阶段探针 → 卡在 `Wire`/u8g2 硬件 I2C 首次传输。 **根因**：Arduino-ESP32 2.0.17（IDF 4.4 旧 I2C 驱动）在总线被模块上电瞬态钳住时，超时中断不触发，初始化**永久阻塞**。用户另一项目同型号 OLED 换 Arduino 3.x （新 I2C 驱动带总线恢复）即正常——与框架版本强相关。 **决策**：本项目固定 2.0.17（LED 等 API 依赖 2.x），于是改用 U8g2 **软件 I2C（GPIO 位拍）**：纯 GPIO 翻转，无驱动、无中断、无锁——机制上不可能卡死， 代价是每帧 40~80 ms（仍远小于 204.8 ms 的块周期，可接受）。

工程思维：在"升级框架（改一切）"与"绕开问题外设驱动（改一处）"之间，评估影响面后选后者。

3. 五键按键：轮询 + 消抖 + 事件队列（`keypad.cpp`）

- 独立 FreeRTOS 任务：10 ms 轮询；**连续 2 拍一致**才确认状态翻转（20 ms 消抖）
- 按下沿产生事件入队（深度 4），主循环 `keypadGetEvent()` 非阻塞取
- **单任务绘制模型**：按键任务只采键发事件，**绝不碰 u8g2/classroom**； 所有界面响应在主循环执行 → 零竞态，不需要互斥锁

4. 交互细节

- 左右键：全局循环切页；LED 页上下键：7 模式列优先循环（左列 4 个 → 右列 3 个），OK 应用
- 教室页：上下选时长（30 s/1 min/10 min），OK 开始；测量中 OK 忽略（防误触）
- 未接 OLED 时按键功能照常（SW I2C 位拍空总线无副作用）

实操环节

1. 按键切换三页，观察 tab 栏反白
2. LED 页用上下键把光标走完 7 个模式（验证列优先顺序），OK 应用后看反白跟随
3. 教室页选 30 s 并启动，切到频闪仪页看实时值（测量继续跑）

检查点

- [ ] 能复述"2 拍一致"消抖为什么能过滤抖动
- [ ] 能说出 SW I2C 与 HW I2C 的优缺点与本项目选型理由

思考题

1. 如果两个任务都要画 OLED，会出现什么问题？有哪几种解法？
2. 队列深度设 4，快速连按 6 次会发生什么？这是缺陷还是特性？

第 8 课 无线网页仪表（30 min）

时间分配

Web 架构 8 min → 协议与前端 7 min → 实操 12 min → 小结 3 min

学习目标

- 理解 AP 模式 + 异步服务器 + WebSocket 的组合
- 能解读 JSON 快照协议并定位前端渲染逻辑

知识讲解

1. 三件套

- **WiFi AP 模式**：开发板自己当热点（`LightSense` / `lightsense`），现场测量无需路由器
- **ESPAsyncWebServer**：异步框架，所有请求非阻塞，不拖慢测量主循环
- **WebSocket**：每报告窗口（≈1 s）主动推送 JSON 快照——比轮询实时且省流量

2. HTTP 路由

路由	功能
`GET /`	内嵌前端页面（三个页签：频闪仪/LED 控制/教室测量）
`GET /led?mode=N`	查询（无参）/切换（带参）LED 模式
`GET /api/classroom`	教室状态 + 历史记录 JSON
`POST /api/classroom/start?label=&dur=`	启动教室测量（dur=30/60/600）
`POST /api/classroom/clear`	清空记录

3. WebSocket 快照（`buildJson`）

每秒一帧：波形包络（256 段 min/max）、对数频谱（256 频带 max-pool，4.9 Hz~5 kHz）、 指标（Pst/SVM/主频/波动深度/光照）、教室进度、判定文案 `zoneNames`、当前 LED 模式 `led`。 前端 Canvas 只做"绘图 + 缩放映射"（对数频谱按组序号线性画，刻度按 log 公式标注）—— **重活留在后端，前端保持轻**。

4. 状态一致性

无论从网页还是 OLED 按键改 LED 模式，快照里都带 `led` 字段 → 前端收到后同步按钮高亮， 两端状态永远一致（双向控制的同步范式）。

实操环节

1. 手机连热点访问 `http://192.168.4.1`，三个页签逐一浏览
2. 网页切 LED 模式 → 观察 OLED 反白同步变化；再从按键切 → 网页按钮同步
3. 打开浏览器 F12 → Network → WS，观察每秒一帧的 JSON

检查点

- [ ] 说出 WebSocket 与 HTTP 轮询的区别及选用理由
- [ ] 在代码里定位 buildJson（webdash.cpp）并对应到网页上的一项数据

思考题

1. 为什么频谱后端先池化成 256 个对数频带，而不是把 1024+ 个 bin 直接发给前端?
2. 如果现场有多个手机同时连着看，架构上需要改什么?

第 9 课 教室测量与数据管理（30 min）

时间分配

流程与数据结构 8 min → NVS 与兼容性设计 7 min → 实操 12 min → 小结 3 min

学习目标

- 掌握"多灯具依次测量"的任务流程设计
- 理解 NVS 键值存储与记录结构的多代兼容策略
- 完成一次完整教室测量并导出 CSV

知识讲解

1. 教室测量流程

选时长（30 s / 1 min / 10 min → 29/58/586 个窗口）→ 打标签（网页输入或自动编号 `灯具1/灯具2/…`）→ 每窗口吞入快照累积均值/最大值 → 时长到自动结算存 NVS → 下一盏。运行中 OLED 显示标签 + 进度条 + 剩余时间。

2. NVS 持久化（`classroom.cpp`）

- NVS = Non-Volatile Storage，Flash 上的键值库，掉电不丢
- 最多 50 条记录，单条 `ClassroomRecord` 74 B：标签 + 时长 + 指标均值/最大值 + 判定码
- **多代兼容**：记录结构历经 64 B → 72 B → 74 B 四个版本；读取时按长度识别旧版， 新字段缺失置 0（前端显示 `--`）。**改结构必须向后兼容，或显式做迁移。**

3. 判定码映射的五处同步（重要工程检查单）

新增/修改判定码要同步：① flicker.cpp 串口文案 ② webdash buildJson `zoneNames` ③ 前端 VNAME 文案 ④ 前端 VCLS 配色 ⑤ oled.cpp ZONE_OLED——漏一处就出显示 bug。

实操环节

1. 网页教室页：选 30 s，对第一盏灯测量，结束后自动命名
2. OLED 教室页再测一盏（自动编号）
3. 导出 CSV，用表格软件打开检查列（含时长列）与判定结果
4. （可选）断电重启 → 记录仍在（NVS 生效验证）

检查点

- [ ] 能画出从"快照"到"一条 NVS 记录"的数据流
- [ ] 说出记录结构兼容性的处理方式

思考题

1. 为什么记录里同时存均值和最大值?
2. 如果要支持 100 条记录，除了容量还要考虑什么？（索引/分页/导出格式）

第 10 课 调试方法论与项目拓展（30 min）

时间分配

案例复盘 12 min → 探针调试法 8 min → 拓展与结业自测 10 min

学习目标

- 从本项目三个真实 bug 中提炼可迁移的调试方法
- 建立"改动检查单"意识
- 明确二次开发方向

知识讲解

案例复盘（本项目真实发生的三个问题）

案例 1：OLED 初始化永久卡死 - 表面看像"接线错误"，实际是 2.0.17 HW I2C 驱动在异常总线下超时不触发 - 方法：**版本相关性排查**——另一项目同硬件 + 新框架正常，锁定"框架版本 × 驱动"矩阵； 再用分阶段探针确认卡点在首次 I2C 传输

案例 2：网页与 OLED 数据全部冻结（按键正常） - 根因链：SW I2C 绘制变慢 → 主循环变慢 → ADC 缓冲耗尽 → 驱动停摆 - 方法：**症状二分**——按键/绘制活而数据死 → 问题在数据链路而非主循环； 再查驱动错误码与缓冲设计

案例 3：LED 页光标只能走左列 4 个模式 - 左右键被"全局切页"占用，而光标按 ±2 同列移动 → 右列永远不可达 - 方法：**交互可达性走查**——每个界面元素都要有一条可达路径；修复时顺带把 网格顺序调成与光标遍历一致（列优先）

探针调试法（嵌入式没有断点时的标准姿势）

1. 在嫌疑路径首尾加分段 `Serial.printf` 探针（打印"到没到 + 关键变量"）
2. 编译烧录，pyserial 抓串口（注意：打开串口会复位板子）
3. 根据最后一行输出定位卡点，逐步收窄
4. 修复后**保留有用的 DBG 行**（如 `DBG ovr=`），它们是下次排障的仪表

改动检查单（改判定码/记录结构/引脚前过一遍）

- 判定码 → 五处映射同步（第 9 课）
- 记录结构 → NVS 兼容性
- 引脚 → strapping/USB/Flash 避让 + readme 引脚表 + config.h 注释

- 换绘制耗时相关的改动 → ADC 缓冲余量是否仍够

拓展方向（选做毕业课题）

1. 串口/蓝牙输出接入上位机，做长期趋势记录
2. 增加 0~80 Hz 波动深度的频谱细化显示
3. 数据记录增加时间戳与教室位置信息
4. 移植到其他 ESP32 板型（练习 platformio 多环境配置）

结业自测清单（全部 ✓ 即结业）

- [ ] 独立完成硬件接线与首次烧录
- [ ] 能解释 Pst^LM / SVM 频段、公式与合格线
- [ ] 完成 5 档 LED 闭环自测并记录数据
- [ ] 完成一次多灯具教室测量 + CSV 导出 + 断电恢复验证
- [ ] 能复述三个调试案例的根因与方法
- [ ] 说出"判定码五处同步"清单

附录

A. 引脚速查表（`src/config.h` 为准）

宏	值	说明
`ADC_PIN`	GPIO1	光信号输入 (ADC1_CH0)
`LED_PIN`	GPIO17 (D7)	LED 闪烁源，220~330 Ω 限流
`KEY_GND_PIN`	GPIO21 (D8)	按键公共端，固件常置低（虚拟 GND）
`OLED_SDA` / `OLED_SCL`	GPIO4/GPIO5	软件 I2C
`KEY_PIN_UP/DOWN/OK/LEFT/RIGHT`	40/41/42/38/39	内部上拉

B. 判定码对照表

码	含义	条件
0	合规 PASS	Pst^LM ≤ 1 且 SVM ≤ 0.4 且光照合规
2	Pst^LM 超标	Pst^LM > 1
3	双指标超标	两者均超标
5	SVM 超标	SVM > 0.4
6	光线过暗	窗口均值 ADC < 400
7	光线太亮	窗口均值 ADC > 3000

（1/4 为旧版遗留码，新数据不再产生。）

C. 常见问题（FAQ）

现象	处理
链接期 undefined reference / WinError 32	OneDrive 锁构建目录：确认 build_dir 重定向到 `C:\pio`
Flash 超过 100%	确认 `board_build.partitions = default_8MB.csv`
串口打开后板子复位	正常现象（DTR 复位），等 1 s 再看输出
网页打不开	确认连的是热点 `LightSense`，地址 `http://192.168.4.1`
OLED 黑屏但按键有效	SW I2C 下正常降级：查 OLED 供电与 SDA/SCL 接线
`DBG ovr=` 持续增长	主循环被严重阻塞：查最近改动是否有耗时操作

D. 术语表

术语	含义
Pst^LM	短期闪烁指示值（照明版），0~80 Hz，≤1 合格
SVM	频闪可见度度量，80~2000 Hz，≤0.4（EU 2024-09）
波动深度	光通量最大最小值差与均值之比
Flicker Index	波形超过均值部分的面积占比
FFT / Hann 窗	快速傅里叶变换 / 抑制频谱泄漏的窗函数
DMA	直接内存访问，外设自动搬运数据不打扰 CPU
LEDC	ESP32 硬件 PWM 外设
NVS	Flash 键值存储，掉电保存
AP 模式	设备自身作为 WiFi 热点
WebSocket	服务端主动推送的全双工通信协议

教程基于 LightSense v1.3（commit a5fff72）编写，引脚/配置以 `src/config.h` 与 `readme.md` 为准。